

Downstream Architecture

From the ingestion core to alerts — ML layer and beyond

Smart India Hackathon · companion to the layered build plan · v1

1. Where the codebase stands

The ingestion core is well built. Rust with PyO3 for the hot path is the right call for a project that has to report a throughput number. Reading Zeek fields *by name* from the `#fields` header rather than by column index is a professional touch that will keep the parsers working across Zeek versions. The module separation is clean, tests exist, and the documentation is better than most production repositories.

Four defects need fixing before any model is trained on this output. Three of them corrupt features silently, which is the worst failure mode: nothing crashes, the numbers look plausible, and the models learn from noise.

1.1 Window features are misaligned with their flows

`sliding_window_features` copies the input, sorts it by timestamp, and returns results in *sorted* order. `extract_flow_features` returns results in *input* order. In `pipeline.py`, the two are merged by index:

```
vec = dict(flow_feats[i])
vec.update(window_feats[i]) # different ordering
```

These only agree when `conn.log` is already sorted by `ts`. Zeek writes a connection record when the connection *closes*, but the `ts` field is when it *opened*. Any long-lived connection therefore appears later in the file than short connections that started after it. On real traffic the file is not sorted, and every window feature is attached to the wrong flow.

FIX

Have the window extractor return the flow's identifier alongside its features and join on that, or sort once in `pipeline.py` before either call and keep a single canonical ordering. Add a regression test using deliberately out-of-order records — the current test data is already sorted, which is why this has not surfaced.

1.2 The sliding window is global, not keyed

This is the most consequential issue. `SlidingWindow` holds every flow in the last N seconds regardless of who sent it. So:

- `unique_dst_ports` is the number of distinct ports contacted by *the entire network*, not by one host. On a busy link this sits in the hundreds permanently and carries no scanning signal.
- `src_ip_entropy` is computed across the whole window rather than across the sources hitting one *destination*. Spoofed-flood detection needs the latter.
- `inter_arrival_mean` and `inter_arrival_stddev` measure the gaps between *all* flows on the wire, not between repeated connections in one conversation. Beacon periodicity is invisible at this granularity.

The README correctly states what these features should indicate. The implementation cannot deliver it, because the aggregation key is missing.

FIX

Replace the single window with a map from entity key to its own window. Three key families are needed: source IP (scanning, flooding), destination IP (source entropy, SYN ratios), and the `(src, dst, dport)` tuple (beaconing). The `SlidingWindow` struct itself is correct and incremental — it just needs to be instantiated per key rather than once globally.

1.3 The first flow in every window gets an absurd rate

`flow_rate` is computed as $n / (\text{span} + 1e-6)$. When a window holds one flow, span is zero and the rate becomes one million. This is visible in the committed `features.jsonl`, where the first row carries `flow_rate: 1000000.0`.

Any gradient-boosted model handed this will discover that "flow_rate equals exactly 1e6" perfectly identifies window boundaries and will learn that instead of learning anything about traffic.

FIX

Divide by the configured window size, not by the observed span, and return zero or a null when fewer than two flows are present. Rate over an unknown span is not a rate.

1.4 The uid join is silently discarded

`join_by_uid` carefully attaches `dns_query`, `ja3_hash`, `ssl_server_name` and friends onto each connection record. That enriched JSON is then passed to `extract_flow_features`, which deserializes it into `FlowRecord` — a struct with none of those fields. Serde drops unknown fields without complaint, so the enrichment evaporates. The DNS, TLS and HTTP blocks in the output actually come from the separate parsed arrays further down the function.

Nothing is broken today, but it is dead work that reads as if it does something, and it will mislead whoever extends the pipeline next.

1.5 Feature-quality gaps worth closing now

Feature	Problem
<code>cipher_encoded</code>	<code>cipher.len() % 16 + 1</code> . This is the length of a string, encoded modulo sixteen. It carries no information about the cipher suite. The README flags it as a placeholder; treat it as a bug rather than a feature.
<code>has_ja3</code>	A boolean saying a fingerprint exists is almost useless — nearly every TLS connection has one. The signal is the fingerprint <i>value</i> and how rare it is against a learned baseline. Keep the hash string.
<code>rcode</code>	Parsed into <code>DnsRecord</code> and never used. NXDOMAIN burst rate per host is often a stronger DGA indicator than the entropy of the domain string itself.
DNS aggregation	All DNS features are per-query. Tunnelling is detected per <i>parent domain</i> : subdomain cardinality, sustained query rate, record-type mix. That aggregation does not exist yet.
Packet timing	<code>conn.log</code> gives totals, not sequences. Packet-size and inter-packet-timing sequences for encrypted-session detection need a Zeek script that emits them, or they must be dropped from scope honestly.

2. The structural change: batch to streaming

The pipeline currently reads whole log files, holds all records in memory, serialises the entire array to JSON, crosses into Rust, and writes an output file. That is precisely the end-of-run report the problem statement rules out in requirement (c).

The good news is that the expensive parts are already correct. `SlidingWindow` is genuinely incremental. The parsers are per-line. What has to change is the driver, not the mathematics.

2.1 Keep both paths, share the code

Do not throw the batch path away. Build the streaming path beside it, and make both call the *same* Rust feature functions.

Path	Used for
Offline	Training-set generation, evaluation, replay experiments. Reads a whole capture, emits labelled feature vectors. This is roughly what exists now.
Online	Live detection. Tails Zeek logs, updates keyed windows incrementally, scores continuously, emits alerts with bounded latency.

TRAIN/SERVE SKEW IS THE CLASSIC WAY THIS PROJECT FAILS

If training features come from one code path and serving features come from another, the model sees different distributions in production than it saw in training, and accuracy collapses for reasons nobody can debug at 2 a.m. The single most valuable property of your current design is that the feature mathematics lives in one Rust crate. Protect that. Neither path should ever compute a feature the other cannot.

2.2 Move the FFI boundary

Today, every record crosses the Rust-Python boundary as JSON: serialise, copy, deserialise, and again on the way back. For a file this is fine. At sustained line rate this becomes the bottleneck, and it defeats the reason for writing the core in Rust at all.

For the streaming path, invert the ownership. Rust holds the window state and never hands it out. Python receives only what it must score, in batches. Concretely:

Rust owns:

Python owns:

Boundary:

log tailing, parsing, keyed windows, baselines,
rule detectors, candidate gating

ML inference, calibration, correlation, API, dashboard

a batch of gated candidates going out,
a batch of scores coming back

This is a small number of crossings per second rather than one per flow.

3. Entity feature views

This is the central architectural idea for everything downstream, and it is what the current single flat vector cannot express.

Your six threat classes do not observe the same thing. They observe different *entities* over different time horizons:

Entity	Horizon	Serves
flow	instant	TLS session scoring, per-connection anomalies
source IP	5s / 60s / 5min	port scanning, flood origination
destination IP	5s / 60s	source-entropy flood detection, amplification
(src, dst, dport)	30min - 6h	beacon periodicity
DNS parent domain	5min / 1h	tunnelling, subdomain fan-out
internal host	rolling baseline	exfiltration, JA3 rarity, destination novelty

A feature vector attached to a flow cannot answer "how many distinct ports has this source touched in the last minute" without the source-keyed view existing. Build these six views as first-class objects in the Rust core. Each detector

subscribes to exactly the views it needs. Nothing computes features it does not use.

The flow-level vector you already emit remains valuable — it becomes the flow view, one of six.

3.1 Memory discipline

The current implementation uses `HashSet` for distinct counts. Under a spoofed flood, the source-keyed map grows one entry per forged address and the process dies. Since demonstrating throughput under attack conditions is a graded requirement, this will surface at exactly the wrong moment.

- HyperLogLog for distinct counts — fixed memory, roughly 2% error, more than adequate for "is this host touching many ports".
- Count-Min Sketch for heavy hitters — top talkers without storing every key.
- A bounded key map with time-based eviction and a hard entry ceiling. When full, evict oldest rather than allocating.
- Streaming entropy over a fixed-size counter table rather than an exact multiset.

This is worth a slide in the presentation. It is the difference between a prototype and something that survives a flood.

4. Detection layer

4.1 Gate before you infer

Do not run models on every flow. Rules run on everything in Rust at line rate and cost microseconds. Models run only on candidates that pass a cheap gate.

```
every flow
-> rule detectors in Rust          (all flows, always)
-> gate: is this DNS?             -> DGA / tunnel models
-> gate: does this have TLS?      -> encrypted-session model
-> gate: outbound + above baseline -> exfiltration model
-> gate: repeated tuple seen 5+ times -> beacon analysis
```

Typical captures gate out well over ninety percent of flows before any model is touched. Your throughput figure ends up governed by Rust, not by Python inference, which is the whole point of the language split.

4.2 What is a rule and what is a model

Detector	Type	Rationale
Port scanning	Rule, Rust	Distinct-destination counting over a keyed window. A model adds nothing and removes explainability.
Volumetric flood	Rule, Rust	Rate against source entropy at the destination. Two thresholds, fully interpretable.
Amplification	Rule, Rust	Response-to-request byte ratio on UDP toward reflector ports.
DNS tunnelling	Rule, Rust	Per-domain cardinality and query-length statistics.
Beaconing	Statistical, Rust	Signal processing on an inter-arrival series. Not a learning problem.
DGA	Model, Python	String classification. Genuinely benefits from learning.
Encrypted session	Model, Python	Multi-feature interaction over metadata. Genuinely benefits.
Exfiltration	Anomaly, Python	Deviation from a learned per-host baseline.

SAY THIS OUT LOUD IN THE PRESENTATION

"We used a statistical test rather than a neural network here because it outperformed the model we tried, runs in microseconds, and can explain every alert." That is a strong answer. Bolting deep learning onto a counting problem is a weak one, and evaluators in this domain will know the difference.

5. The ML detectors in detail

MODEL 1

DGA classification

Why the current features are not enough

Query entropy, length and label count will reach perhaps eighty percent on classic random-character families and then stop. They fail completely on dictionary-based generators, which assemble domains from real English words. Those score *lower* entropy than many legitimate content-delivery hostnames, so an entropy threshold has the sign backwards for that whole family.

Input representation

Feed the model the domain string itself, not a scalar summary of it. Two options, both viable:

- **Gradient boosting over character n-grams.** TF-IDF over character bigrams and trigrams, concatenated with the scalar features you already compute. Trains in seconds, runs in microseconds, and is straightforward to explain.
- **Character-level convolutional network.** Embed characters, apply small convolutions, pool, classify. Stronger on dictionary families. Heavier to train and to justify.

Start with the first. Only reach for the second if evaluation shows a specific family it cannot handle.

Add the behavioural signal

The strongest DGA indicator is often not the string at all. Infected hosts generate hundreds of candidate domains and most do not resolve. A burst of NXDOMAIN responses from a single host inside a short window is a powerful, cheap, per-host feature — and you already parse `rcode` without using it.

Combine both: string score from the model, NXDOMAIN burst rate from the host view, fused into one finding. String evidence alone produces false positives on content-delivery hostnames; the behavioural signal disambiguates them.

Training data

Legitimate domains from a large public ranking list; generated domains from published algorithm implementations across multiple families. Split by *family*, not randomly — hold out entire families from training so your reported score reflects generalisation to unseen generators rather than memorisation of the ones you trained on. This distinction is worth stating explicitly in the documentation; it is what separates a real evaluation from a flattering one.

MODEL 2

Encrypted-session classification

The feature that matters most

Replace `has_ja3` with the fingerprint value and a rarity score. During the learning phase, accumulate a frequency table of every JA3 hash observed. At detection time, the feature is how unusual this client's fingerprint is relative to that baseline, and whether this specific host has ever presented it before.

This works because a browser, a scripting-language HTTP library, and a command-line tool each offer different cipher suites in a different order, producing distinct fingerprints. A host that has presented a browser fingerprint for a week and suddenly presents a scripting-library fingerprint to an unfamiliar destination is the signal. No payload is involved at any point.

Full feature set

Feature	Source
JA3 global rarity	frequency table from baseline
JA3 novelty for this host	host view
JA3S rarity	frequency table
SNI present / absent	<code>ssl.log</code>
SNI entropy and length	computed
Certificate validity window	<code>x509.log</code> — not yet parsed
Self-signed indicator	<code>x509.log</code> — not yet parsed
TLS version	already encoded
Mean packet size each direction	bytes divided by packets
Duration, byte ratio	flow view

Two of these require an `x509.log` parser, which does not exist yet. It follows the same pattern as the four you have written and is perhaps an hour of work for meaningful signal.

BE HONEST ABOUT THE SEQUENCE FEATURES

The literature on this problem leans heavily on packet-size and inter-arrival *sequences*. `conn.log` gives totals, not sequences, so mean packet size is a weak proxy. Either add a Zeek script that emits per-packet sizes for the first several packets of each TLS connection, or state plainly in the documentation that sequence modelling is out of scope for this prototype and explain why. Both are defensible. Claiming sequence analysis you have not implemented is not.

MODEL 3

Exfiltration anomaly detection

Why supervised learning is the wrong tool here

You cannot enumerate what exfiltration looks like — it looks like whatever a large upload looks like. What you can characterise is what *this host normally does*. So the model is unsupervised, scoring deviation from a per-host baseline.

Baseline contents, per internal host

- Distribution of outbound bytes per hour, kept as robust statistics rather than mean and standard deviation, since a single backup would otherwise poison the baseline permanently.
- Set of previously contacted destinations, so novelty is measurable.
- Typical active hours.
- Typical outbound-to-inbound ratio.

Scoring

Robust z-score on outbound volume, plus destination novelty, plus time-of-day deviation, combined either by a small isolation forest or by a weighted sum you tuned on validation data. Prefer the simpler option unless it demonstrably underperforms.

THIS DETECTOR WILL PRODUCE YOUR WORST FALSE-POSITIVE RATE

Cloud backup, video calls, and large uploads all look exactly like exfiltration on a volume anomaly. Tune this one for precision over recall and say so explicitly. A detector that catches sixty percent of exfiltration with two false alarms per day is operationally useful; one that catches ninety percent with two hundred false alarms per day gets switched off in week one. State that trade-off before a judge asks about it.

MODEL 4

Beacon periodicity

Not a learned model, but it belongs here because it operates on a series rather than a record.

For each `(src, dst, dport)` tuple with at least a handful of observations, take the connection timestamps and compute the inter-arrival series. Then:

- **Coefficient of variation** — standard deviation over mean. Near zero means near-perfect regularity.
- **Autocorrelation at the dominant lag** — catches beacons whose jitter defeats a simple variance test.
- **Spectral peak strength** — a strong single frequency in the arrival series.
- **Payload-size consistency** — automated check-ins transfer near-identical byte counts each time. Humans do not.

Combine into one score. Then sweep the jitter parameter on your own emulator and report the breaking point: *"holds to forty percent jitter, degrades beyond"*. That single sentence is worth more than any accuracy figure, because it shows you characterised your detector rather than merely ran it.

The confounders

Time synchronisation, package update checks and monitoring heartbeats are all perfectly periodic and entirely innocent. If your benign traffic contains none of them, this detector will report zero false positives and that number will be meaningless. Plant them deliberately, then report precision against them. That is the credible version of this result.

6. Training and evaluation infrastructure

6.1 The training pipeline

```
labelled pcap + scenario manifest
-> Zeek
-> Rust parsers      [same code as online]
-> entity feature views [same code as online]
-> label join on manifest
-> per-detector training set
-> train, cross-validate, calibrate
-> model artefact + calibration curve + metrics
-> registry
```

Every model artefact ships with the feature schema version it was trained against. On load, the serving side checks that version against what the feature store produces and refuses to start on a mismatch. This single check prevents the most common and most confusing production failure in ML systems.

6.2 Calibration

Raw model outputs are not probabilities, and a rule's boolean is not comparable to a gradient-boosted score. The problem statement requires a confidence value; that value must mean the same thing regardless of which detector produced it.

Fit an isotonic or Platt calibration per detector on held-out data, and store the calibration curve with the model. After calibration, a stated confidence of 0.8 should correspond to roughly eighty percent of such alerts being genuine. Show the reliability diagram in your documentation — almost no competing team will have one, and it is direct evidence that your confidence scores are real rather than decorative.

6.3 Evaluation

Report per threat class: precision, recall, F1, and false positives per hour on purely benign traffic. Do not report overall accuracy; with rare positives it is dominated by the negative class and a detector that fires on nothing scores extremely well. Anyone familiar with the domain will notice immediately.

Also report the operating point you chose and why. A precision-recall curve with a marked threshold and one sentence of justification demonstrates engineering judgement in a way a single number cannot.

7. Fusion layer

Eight detectors emitting findings independently produce an unusable alert stream. Four stages sit between findings and alerts.

Calibrate

Apply the stored curve. Raw score becomes comparable confidence.

Assign severity

Severity is not confidence. Combine confidence with the inherent seriousness of the threat class and, if modelled, the importance of the asset. A moderate-confidence exfiltration finding on a critical server outranks a high-confidence scan against a spare machine.

Deduplicate

Suppress repeats for the same entity, class and evidence pattern inside a time window. Emit one alert carrying an occurrence count and a first-seen timestamp, not sixty identical rows.

Correlate

Group alerts sharing a host inside a window into an incident, ordered along the kill chain: reconnaissance, then command and control, then exfiltration. An incident scores higher than the sum of its parts, because the sequence is itself evidence.

This layer is the highest value per line of code in the entire project, and it is the one most competing teams will not build at all.

8. Serving, storage and interface

8.1 Model serving

Models load once at startup into a scoring service that accepts batches. Every response carries the model version, which is stamped onto the alert. Two properties matter: the service must never block the rule path, and a model failure must degrade to rules-only rather than halting detection. Log the degradation, keep detecting.

8.2 Alert fan-out

Each alert goes to two independent consumers in parallel — a WebSocket channel for the live dashboard, and a batched writer for PostgreSQL. The live path never waits on the durable path. Routing alerts through the database before display converts your streaming system into a micro-batch system and forfeits requirement (c).

Stamp two timestamps on every alert: the triggering packet's time and the emission time. The difference is your detection latency, which you are required to report.

8.3 Storage

An `alerts` table with evidence as JSONB, an `incidents` table, and a join between them. Index on timestamp, source IP and threat class — those three cover essentially every query the dashboard and the analyst layer will issue. Batch the inserts; one write per alert becomes the bottleneck during a flood, which is exactly when you least want one.

8.4 Dashboard

Six panels, in descending order of how much they matter on demo day:

1. Live alert feed over WebSocket, filterable by class, severity and host.
2. Evidence drill-down — click any alert, see the exact features, their values, and the thresholds they crossed. This directly answers the supporting-evidence requirement and is the single most convincing thing on the screen.
3. Incident view showing correlated activity on one host as an ordered narrative.
4. Throughput panel: live flows per second, packets per second, megabits per second.
5. Latency panel: distribution of packet-to-alert delay with median and tail marked.
6. Host timeline, which is how an analyst actually investigates.

8.5 The RAG analyst layer

Strictly downstream, strictly read-only against the alert store. It explains an alert in plain language from its evidence fields, narrates a correlated incident, and answers natural-language questions over persisted alerts.

Hard boundaries: it never influences detection, never modifies a score, and is never on the critical path for emitting an alert. Every statement it produces must trace back to a specific stored evidence field. The system must pass every requirement in the problem statement with this layer switched off — be ready to demonstrate that.

PREPARE FOR THE NETWORK-ACCESS QUESTION

A judge will ask whether the language model has network access and whether that breaks the read-only constraint. Calling a hosted API is egress from the monitoring enclave — defensible, but it invites argument you do not need. A locally hosted model removes the question entirely and makes for a cleaner demo. If you use a hosted API, document the boundary explicitly and show that the ingest path remains one-directional regardless.

9. Immediate work queue

#	Task	Why it is ordered here
1	Fix the window/flow ordering misalignment	Every window feature is currently attached to the wrong flow on unsorted logs. Nothing downstream is trustworthy until this is corrected.
2	Fix <code>flow_rate</code> division	The 1e6 artefact will be learned by any model trained on current output.
3	Key the sliding windows by entity	Scanning, flooding and beaconing detection are all impossible without this. It is the largest single change and it blocks four detectors.
4	Build the six entity views	The interface every detector reads from. Freeze it early.
5	Bound the memory with sketches	Do this while the views are being written, not as a later optimisation. Retrofitting is painful.
6	Streaming driver beside the batch driver	Both call the same feature code. Requirement (c) depends on this existing.
7	Rule detectors in Rust	Four of eight detectors, all cheap, all explainable. Fastest path to visible results.
8	<code>x509.log</code> parser; keep JA3 values and build the rarity table	Unblocks the encrypted-session model.
9	Per-domain DNS aggregation; wire up <code>rcode</code>	Unblocks tunnelling detection and materially improves DGA.
10	Train DGA and encrypted-session models	Only now, once the features they need actually exist.
11	Calibration and fusion	Turns eight detectors into one coherent system.
12	Alert bus, storage, dashboard	Can be built against fixtures in parallel with everything above.

What to do first

Items one and two are each a few lines and can be done today. Item three is the substantial one — a day or two of careful work, and it unblocks more of the project than anything else on the list. Do not start training models until item nine lands, because models trained on the current feature set would have to be retrained from scratch afterwards.

WORTH SAYING PLAINLY

The foundation here is better than most teams will have at this stage. Header-aware parsing, a Rust hot path, real tests and honest documentation are not common in hackathon repositories. The defects above are ordinary consequences of building the batch version first — which was the right order to build in. Fix the four, key the windows, and the rest of the system has a solid base to sit on.